



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Práctica Guiada 12: Preparación Intensiva para Examen I - Sistema de Matrícula Académica "EduConnect" (Spring Boot 3 + H2 Database + Angular 19)

Ficha Técnica de la Práctica

- **Objetivo Principal:** Servir como marco de entrenamiento semiguizado y preparación integral para el **Examen Parcial I**, reforzando la construcción desacoplada de aplicaciones Full-Stack en **Spring Boot 3** (Back-End) y **Angular 19 Standalone** (Front-End) utilizando una base de datos en memoria **H2 Database**.
- **Tiempo Estimado:** 2 horas y 30 minutos (150 minutos).
- **Modalidad:** Trabajo Autónomo Individual / Práctica Guiada de Laboratorio.

- **Cobertura de Contenidos:** Temas 1 a 4 del programa del curso (Fundamentos Web, Setup/Git, Back-End RESTful/JPA/Mockito y Front-End Angular/Signals).
- **Motor de Base de Datos:** H2 Database en Memoria (`jdbc:h2:mem:educonnectdb`).
- **Estructura del Proyecto:**
 - Backend: Spring Boot 3 (Java 17+) en subcarpeta `educonnect-backend`
 - Frontend: Angular 19 Standalone en subcarpeta `educonnect-frontend`
 - Recursos SQL Iniciales: Disponibles en `resources/sql/practica_12/`

Introducción y Caso de Negocio: EduConnect

La institución educativa **EduConnect** requiere modernizar su plataforma de gestión académica para procesar las solicitudes de matrícula de asignaturas universitarias de forma eficiente.

El sistema debe permitir a los estudiantes matricular materias asociando a un **Estudiante**, un **Profesor** impartidor y un **Curso**, registrando el código único de matrícula (ej. MAT-2026-0101), la cantidad de créditos, el periodo lectivo, la fecha de registro y el estado actual (ACTIVA, COMPLETADA, CANCELADA).

Para asegurar que los estudiantes puedan desarrollar la práctica en cualquier laboratorio o computadora personal sin depender de una instalación previa de SQL Server local, esta práctica utiliza **H2 Database en memoria**. Se proveen los archivos de inicialización ANSI SQL (`schema.sql` y `data.sql`) en la carpeta de recursos de inicio `resources/sql/practica_12/`.

Parte 1: Configuración e Inicialización de Base de Datos H2 (En Memoria)

En esta sección configurará la persistencia en memoria dentro del proyecto Back-End en Spring Boot 3 (`educonnect-backend`).

1.1 Archivos SQL Iniciales (Kickoff Scripts)

Copie los archivos de la carpeta de recursos provista `resources/sql/practica_12/` hacia la carpeta de recursos de su proyecto Back-End:

- `resources/sql/practica_12/schema.sql` ->
`educonnect-backend/src/main/resources/schema.sql`

`resources/sql/practica_12/data.sql` ->
`educonnect-backend/src/main/resources/data.sql`

schema.sql: Contiene la definición DDL de las tres tablas relacionales (`profesor`, `estudiante` y `matricula_curso`) con llaves primarias autoincrementables y restricciones de llave foránea (`CONSTRAINT fk_matricula_estudiante` y `CONSTRAINT fk_matricula_profesor`).
- **data.sql:** Contiene las sentencias `INSERT INTO` con los datos semilla iniciales de profesores, estudiantes y registros de matrícula para el arranque de la aplicación.

1.2 Dependencia Maven (`pom.xml`)

Verifique que el archivo `pom.xml` contenga la dependencia de H2 Database:

```
<dependency>
  <groupId>com.h2database</groupId>
  <artifactId>h2</artifactId>
  <scope>runtime</scope>
</dependency>
```

1.3 Configuración en `application.properties`

Abra `src/main/resources/application.properties` y configure el motor H2, la consola web y la ejecución automática de los scripts SQL deshabilitando la generación DDL de Hibernate:

```
# Configuración H2 Database En Memoria
spring.datasource.url=jdbc:h2:mem:educonnectdb;DB_CLOSE_DELAY=-1
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
```

```
spring.datasource.password=  
spring.jpa.database-platform=org.hibernate.dialect.H2Dialect  
  
# Desactivar auto-ddl de Hibernate para schema.sql  
spring.jpa.hibernate.ddl-auto=none  
  
# H2 Console y ejecucion de scripts SQL  
spring.h2.console.enabled=true  
spring.sql.init.mode=always  
  
# Puerto de ejecucion  
server.port=8080
```

Parte 2: Guía de Desarrollo Back-End (educonnect-backend)

Estructure el proyecto Spring Boot 3 respetando la arquitectura por capas limpia:

```
educonnect-backend/src/main/java/com/educonnect/  
  controller/  
    MatriculaCursoController.java  
  dto/  
    CrearMatriculaDTO.java  
    MatriculaCursoDTO.java  
  exception/  
    GlobalExceptionHandler.java  
  model/  
    Estudiante.java  
    MatriculaCurso.java  
    Profesor.java  
  repository/  
    MatriculaCursoRepository.java  
  service/  
    MatriculaCursoService.java
```

2.1 Entidades JPA (com.educonnect.model)

Cree las tres clases JPA representando las tablas de la base de datos H2:

- **Profesor.java:**
- Campos: Long id, String nombre, String especialidad, Boolean activo.
- **Estudiante.java:**
- Campos: Long id, String carnet, String nombreCompleto, String correo, String carrera.
- **MatriculaCurso.java:**
- Campos: Long id, String codigoMatricula, @ManyToOne Estudiante estudiante, @ManyToOne Profesor profesor, String nombreCurso, Integer creditos, String periodo, String estado, BigDecimal montoArancel, LocalDateTime fechaMatricula.

Fragmento de Referencia (MatriculaCurso.java):

```
package com.educonnect.model;

import jakarta.persistence.*;
import java.math.BigDecimal;
import java.time.LocalDateTime;

@Entity
@Table(name = "matricula_curso")
public class MatriculaCurso {

    @Id
    @GeneratedValue(
        strategy = GenerationType.IDENTITY
    )
    private Long id;

    @Column(
        name = "codigo_matricula",
        nullable = false,
        unique = true
    )
    private String codigoMatricula;

    @ManyToOne(optional = false)
    @JoinColumn(
        name = "estudiante_id",
        nullable = false
    )
```

```

    private Estudiante estudiante;

    @ManyToOne(optional = false)
    @JoinColumn(
        name = "profesor_id",
        nullable = false
    )
    private Profesor profesor;

    @Column(name = "nombre_curso", nullable = false)
    private String nombreCurso;

    @Column(nullable = false)
    private Integer credits;

    @Column(nullable = false)
    private String periodo;

    @Column(nullable = false)
    private String estado;

    @Column(name = "monto_arancel", nullable = false)
    private BigDecimal montoArancel;

    @Column(
        name = "fecha_matricula",
        nullable = false
    )
    private LocalDateTime fechaMatricula;

    // Constructores, Getters y Setters
}

```

2.2 Objetos de Transferencia de Datos (`com.educonnect.dto`)

Defina los DTOs desacoplados de las entidades relacionales:

1. **MatriculaCursoDTO**: Para enviar las matrículas a Angular.
2. Campos: `Long id`, `String codigoMatricula`, `String nombreEstudiante`, `String carnetEstudiante`, `String nombreProfesor`, `String nombreCurso`, `Integer credits`, `String periodo`, `String estado`, `BigDecimal montoArancel`, `LocalDateTime fechaMatricula`.

3. **CrearMatriculaDTO**: Para recibir la solicitud desde el cliente.
4. Campos: Long estudianteId, Long profesorId, String nombreCurso, Integer credits, String periodo, BigDecimal montoArancel.

2.3 Repositorio JPA (`com.educonnect.repository`)

Cree `MatriculaCursoRepository.java` extendiendo de `JpaRepository<MatriculaCurso, Long>` e incorpore métodos de consulta por estado:

```
package com.educonnect.repository;

import com.educonnect.model.MatriculaCurso;
import org.springframework.data.jpa
    .repository.JpaRepository;
import org.springframework.stereotype
    .Repository;
import java.util.List;

@Repository
public interface MatriculaCursoRepository
    extends JpaRepository<MatriculaCurso, Long> {

    List<MatriculaCurso>
        findByEstadoOrderByFechaMatriculaDesc(
            String estado
        );

    List<MatriculaCurso>
        findAllByOrderByFechaMatriculaDesc();
}
```

2.4 Servicio de Negocio (`com.educonnect.service`)

Implemente `MatriculaCursoService.java` con las siguientes reglas de negocio: 1. `obtenerTodas()`: Retorna lista de DTOs ordenada por fecha descendente. 2. `obtenerPorEstado(String estado)`: Retorna matrículas filtradas por estado (ACTIVA, COMPLETADA, CANCELADA). 3. `crearMatricula(CrearMatriculaDTO dto)`: * Genera el código consecutivo dinámico

MAT-2026- + número aleatorio o timestamp. * Valida la existencia del estudiante y profesor. Si el profesor no está activo (`activo == false`), lanza `IllegalArgumentException("El profesor seleccionado no se encuentra activo.")`. * Asigna el estado inicial `ACTIVA` y la fecha actual `LocalDateTime.now()`. 4. `actualizarEstado(Long id, String nuevoEstado)`: Cambia el estado de la matrícula.

2.5 Controlador RESTful (`com.educonnect.controller`)

Cree `MatriculaCursoController.java` mapeado en `/api/v1/matriculas` habilitando CORS para Angular (`http://localhost:4200`):

```
package com.educonnect.controller;

import com.educonnect.dto.CrearMatriculaDTO;
import com.educonnect.dto.MatriculaCursoDTO;
import com.educonnect.service.MatriculaCursoService;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/v1/matriculas")
@CrossOrigin(origins = "http://localhost:4200")
public class MatriculaCursoController {

    private final MatriculaCursoService service;

    public MatriculaCursoController(
        MatriculaCursoService service) {
        this.service = service;
    }

    @GetMapping
    public ResponseEntity<List<MatriculaCursoDTO>>
        obtenerTodas() {
        return ResponseEntity.ok(
            service.obtenerTodas()
        );
    }
}
```

```

    @GetMapping("/estado/{estado}")
    public ResponseEntity<List<MatriculaCursoDTO>>
    obtenerPorEstado(
        @PathVariable String estado) {
        return ResponseEntity.ok(
            service.obtenerPorEstado(estado)
        );
    }

    @PostMapping
    public ResponseEntity<MatriculaCursoDTO>
    crearMatricula(
        @RequestBody CrearMatriculaDTO dto) {
        return ResponseEntity.status(HttpStatus.CREATED)
            .body(service.crearMatricula(dto));
    }

    @PatchMapping("/{id}/estado")
    public ResponseEntity<MatriculaCursoDTO>
    actualizarEstado(
        @PathVariable Long id,
        @RequestParam String nuevoEstado) {
        return ResponseEntity.ok(
            service.actualizarEstado(id, nuevoEstado)
        );
    }
}

```

Parte 3: Excepciones RFC 7807 y Pruebas Unitarias Mockito

3.1 Manejo de Excepciones Centralizado (`com.educonnect.exception`)

Implemente un `@RestControllerAdvice` retornando la norma internacional **RFC 7807 Problem Details** (`ProblemDetail` de Spring Boot 3):

```

package com.educonnect.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ProblemDetail;
import org.springframework.web.bind.annotation

```

```

    .ExceptionHandler;
import org.springframework.web.bind.annotation
    .RestControllerAdvice;

import java.net.URI;
import java.time.Instant;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(
        IllegalArgumentException.class
    )
    public ProblemDetail handleIllegalArgumentException(
        IllegalArgumentException ex) {
        ProblemDetail problem =
            ProblemDetail.forStatusAndDetail(
                HttpStatus.BAD_REQUEST,
                ex.getMessage()
            );
        problem.setTitle(
            "Solicitud Invalida de Matricula"
        );
        problem.setType(URI.create(
            "https://educonnect.ac.cr/errors/bad-request"
        ));
        problem.setProperty(
            "timestamp", Instant.now()
        );
        return problem;
    }

    @ExceptionHandler(RuntimeException.class)
    public ProblemDetail handleRuntimeException(
        RuntimeException ex) {
        ProblemDetail problem =
            ProblemDetail.forStatusAndDetail(
                HttpStatus.NOT_FOUND,
                ex.getMessage()
            );
        problem.setTitle("Recurso No Encontrado");
        problem.setType(URI.create(
            "https://educonnect.ac.cr/errors/not-found"
        ));
        problem.setProperty(
            "timestamp", Instant.now()
        );
    }
}

```

```

    );
    return problem;
}
}

```

3.2 Prueba Unitaria con JUnit 5 y Mockito

Cree la prueba unitaria en `src/test/java/com/educonnect/service/MatriculaCursoServiceTest.java` aislando el repositorio mediante `@Mock` y `@InjectMocks`:

```

package com.educonnect.service;

import com.educonnect.dto.CrearMatriculaDTO;
import com.educonnect.dto.MatriculaCursoDTO;
import com.educonnect.model.Estudiante;
import com.educonnect.model.MatriculaCurso;
import com.educonnect.model.Profesor;
import com.educonnect.repository
    .EstudianteRepository;
import com.educonnect.repository
    .MatriculaCursoRepository;
import com.educonnect.repository
    .ProfesorRepository;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.extension
    .ExtendWith;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.junit.jupiter
    .MockitoExtension;

import java.math.BigDecimal;
import java.util.Optional;

import static org.junit.jupiter.api.Assertions.*;
import static org.mockito.ArgumentMatchers.any;
import static org.mockito.Mockito.*;

@ExtendWith(MockitoExtension.class)
class MatriculaCursoServiceTest {

```

```

@Mock
private MatriculaCursoRepository matriculaRepository;

@Mock
private EstudianteRepository estudianteRepository;

@Mock
private ProfesorRepository profesorRepository;

@InjectMocks
private MatriculaCursoService service;

@Test
void crearMatricula_Exito() {
    // Arrange
    CrearMatriculaDTO dto =
        new CrearMatriculaDTO();
    dto.setEstudianteId(1L);
    dto.setProfesorId(1L);
    dto.setNombreCurso("Desarrollo Web");
    dto.setCreditos(4);
    dto.setPeriodo("II-2026");
    dto.setMontoArancel(
        new BigDecimal("45000.00")
    );

    Estudiante estudiante = new Estudiante();
    estudiante.setId(1L);
    estudiante.setNombreCompleto(
        "Valeria Monge"
    );

    Profesor profesor = new Profesor();
    profesor.setId(1L);
    profesor.setNombre("Dr. Armando Ramos");
    profesor.setActivo(true);

    when(estudianteRepository.findById(1L))
        .thenReturn(Optional.of(estudiante));
    when(profesorRepository.findById(1L))
        .thenReturn(Optional.of(profesor));
    when(matriculaRepository
        .save(any(MatriculaCurso.class)))
        .thenReturn(inv ->
            inv.getArgument(0));
}

```

```

    // Act
    MatriculaCursoDTO resultado =
        service.crearMatricula(dto);

    // Assert
    assertNotNull(resultado);
    assertTrue(resultado.getCodigoMatricula()
        .startsWith("MAT-2026-"));
    assertEquals("ACTIVA",
        resultado.getEstado());
    verify(matriculaRepository, times(1))
        .save(any(MatriculaCurso.class));
}
}

```

Parte 4: Guía de Desarrollo Front-End (`educonnect-frontend`)

Inicialice la aplicación cliente ejecutando en la terminal:

```
ng new educonnect-frontend --standalone --style=css --ssr=false
```

4.1 Configuración Global (`app.config.ts` y `environment.ts`)

1. En `src/environments/environment.ts`:

```

export const environment = {
  production: false,
  apiUrl: 'http://localhost:8080/api/v1'
};

```

1. En `src/app/app.config.ts`:

```

import { ApplicationConfig,
  provideZoneChangeDetection } from '@angular/core';

```

```
import { provideRouter } from '@angular/router';
import { provideHttpClient,
  withFetch } from '@angular/common/http';
import { routes } from '../app.routes';

export const appConfig: ApplicationConfig = {
  providers: [
    provideZoneChangeDetection(
      { eventCoalescing: true }
    ),
    provideRouter(routes),
    provideHttpClient(withFetch())
  ]
};
```

4.2 Modelos e Interfaces (src/app/models/matricula.model.ts)

Cree las interfaces TypeScript:

```
export interface MatriculaCurso {
  id?: number;
  codigoMatricula: string;
  nombreEstudiante: string;
  carnetEstudiante: string;
  nombreProfesor: string;
  nombreCurso: string;
  creditos: number;
  periodo: string;
  estado: 'ACTIVA' | 'COMPLETADA' | 'CANCELADA';
  montoArancel: number;
  fechaMatricula: string;
}

export interface CrearMatriculaPayload {
  estudianteId: number;
  profesorId: number;
  nombreCurso: string;
  creditos: number;
  periodo: string;
  montoArancel: number;
}
```

4.3 Servicio Angular (src/app/services/matricula.service.ts)

Implemente la comunicación asíncrona mediante `HttpClient` e inyección de dependencias `inject()`:

```
import { inject, Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { environment }
  from '../../environments/environment';
import { CrearMatriculaPayload,
  MatriculaCurso }
  from '../models/matricula.model';

@Injectable({
  providedIn: 'root'
})
export class MatriculaService {
  private http = inject(HttpClient);
  private baseUrl =
    `${environment.apiUrl}/matriculas`;

  getMatriculas(): Observable<MatriculaCurso[]> {
    return this.http.get<MatriculaCurso[]>(
      this.baseUrl
    );
  }

  getMatriculasPorEstado(
    estado: string
  ): Observable<MatriculaCurso[]> {
    return this.http.get<MatriculaCurso[]>(
      `${this.baseUrl}/estado/${estado}`
    );
  }

  crearMatricula(
    payload: CrearMatriculaPayload
  ): Observable<MatriculaCurso> {
    return this.http.post<MatriculaCurso>(
      this.baseUrl, payload
    );
  }

  actualizarEstado(
```

```

    id: number, nuevoEstado: string
  ): Observable<MatriculaCurso> {
    return this.http.patch<MatriculaCurso>(
      `${this.baseUrl}/${id}/estado` +
      `?nuevoEstado=${nuevoEstado}`, {}
    );
  }
}

```

4.4 Dashboard Reactivo con Signals (`MatriculaListComponent`)

Genere el componente de listado: `ng g c components/matricula-list --standalone`.

Implemente **Angular Signals** (`signal()`, `computed()`) para almacenar el catálogo de matrículas y realizar un filtrado dinámico por estado (TODAS, ACTIVA, COMPLETADA, CANCELADA).

Lógica TypeScript (`matricula-list.component.ts`):

```

import { Component, inject, OnInit,
  signal, computed } from '@angular/core';
import { CommonModule } from '@angular/common';
import { MatriculaService }
  from '../../services/matricula.service';
import { MatriculaCurso }
  from '../../models/matricula.model';

@Component({
  selector: 'app-matricula-list',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './matricula-list.component.html',
  styleUrls: ['./matricula-list.component.css']
})
export class MatriculaListComponent implements OnInit {
  private matriculaService = inject(MatriculaService);

  // Writable Signals
  matriculas = signal<MatriculaCurso[]>([]);
  filtroEstado = signal<string>('TODAS');

  // Computed Signal para filtrado reactivo
  matriculasFiltradas = computed(() => {

```

```

    const estado = this.filtroEstado();
    if (estado === 'TODAS') {
        return this.matriculas();
    }
    return this.matriculas()
        .filter(m => m.estado === estado);
});

ngOnInit(): void {
    this.cargarMatriculas();
}

cargarMatriculas(): void {
    this.matriculaService.getMatriculas()
        .subscribe({
            next: (data) => this.matriculas.set(data),
            error: (err) =>
                console.error('Error al cargar:', err)
        });
}

cambiarFiltro(nuevoEstado: string): void {
    this.filtroEstado.set(nuevoEstado);
}

avanzarEstado(item: MatriculaCurso): void {
    if (!item.id) return;
    const siguienteEstado =
        item.estado === 'ACTIVA' ?
        'COMPLETADA' : 'CANCELADA';
    this.matriculaService.actualizarEstado(
        item.id, siguienteEstado
    ).subscribe({
        next: () => this.cargarMatriculas(),
        error: (err) =>
            alert('Error al actualizar: ' + err.message)
    });
}
}

```

4.5 Componente Formulario (MatriculaFormComponent)

Genere el formulario: `ng g c components/matricula-form --standalone`.

Incorpore `FormsModule` con data binding bidireccional `[(ngModel)]` para capturar la nueva solicitud y redirigir a `/matriculas` al guardar con éxito.

4.6 Enrutamiento (`app.routes.ts`)

Configure la navegación entre las dos vistas principales:

```
import { Routes } from '@angular/router';
import { MatriculaListComponent }
  from './components/matricula-list/matricula-list.component';
import { MatriculaFormComponent }
  from './components/matricula-form/matricula-form.component';

export const routes: Routes = [
  { path: '', redirectTo: 'matriculas', pathMatch: 'full' },
  { path: 'matriculas', component: MatriculaListComponent },
  { path: 'nueva-matricula', component: MatriculaFormComponent }
];
```

Parte 5: Versionamiento y Commits Semánticos en Git

Durante el desarrollo de la práctica, ejecute al menos 5 commits semánticos en su repositorio personal:

1. `feat(setup): initialize spring boot project with H2 DB scripts`
2. `feat(backend): implement JPA entities and DTOs for EduConnect`
3. `feat(backend): add MatriculaCursoService and REST controller`
4. `test(backend): add Mockito unit test for crearMatricula`
5. `feat(frontend): implement Angular 19 MatriculaListComponent with Signals`

Distribución Recomendada del Tiempo (150 Minutos)

Bloque de Tiempo	Fase de Desarrollo	Duración Sugerida
Bloque 1	Copiar kickoff SQL (<code>schema.sql</code> , <code>data.sql</code>) y configurar H2 (<code>application.properties</code>)	15 min
Bloque 2	Backend: Entidades, DTOs, Repository, Service y Controller RESTful	45 min
Bloque 3	Backend: Manejador RFC 7807 y Prueba Unitaria Mockito	15 min
Bloque 4	Setup Angular 19, <code>environment.ts</code> , Models y <code>MatriculaService</code>	15 min
Bloque 5	Frontend: <code>MatriculaListComponent</code> con Signals y <code>MatriculaFormComponent</code>	45 min
Bloque 6	Integración Full-Stack, pruebas end-to-end y Commits Git	15 min

Rúbrica de Evaluación y Verificación Autónomo (100 Puntos)

Criterio	Puntos	Descripción de Verificación
Persistencia H2 & Arquitectura Back-End	35 pts	Mapeo JPA correcto de 3 entidades en H2 en memoria, controladores RESTful con CORS y servicio de negocio funcional.
Front-End Angular 19 SPA & Signals	35 pts	Componentes Standalone enrutados, reactividad mediante Angular Signals (<code>signal()</code> , <code>computed()</code>) y consumo HTTP con <code>HttpClient</code> .
Excepciones RFC 7807 & Pruebas Mockito	15 pts	<code>@RestControllerAdvice</code> formateando errores en Problem Details y prueba unitaria en Mockito ejecutando sin fallos (<code>mvn test</code>).
Versionamiento & Commits Semánticos	15 pts	Historial de Git estructurado con mensajes de commit semánticos periódicos.